Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna

Practica 3: Calidad de la visualización. Checks

Visualización de Datos

Autor: Cheuk Kelly Ng Pante (alu0101364544@ull.edu.es)

Fecha: 1 de marzo de 2026

Índice general

1. Utilización del laboratorio de checks **1**
1.1. Análisis de `test_checks.py` 1
1.2. Análisis de `definitions.py` 1
1.3. Lanzamiento y materialización de los assets 1

2. Análisis de la práctica anterior **3**

3. Verificación de los checks: casos de fallo **7**

4. Enlace al repositorio de GitHub **9**

1. Utilización del laboratorio de checks

1.1. Análisis de `test_checks.py`

Lo que hace el asset `islas_raw` es cargar los datos del CSV y añade intecionalmente una "fila sucia" (con el valor "tenerife" en minúsculas) para que el check falle. El check `check_estandarizacion_islas` comprueba que no existan errores de formato (mayúsculas/minúsculas). Lo hace comparando la cantidad de categorías únicas originales con las categorías únicas que resultan al normalizar el texto, poniendo la primera letra en mayúscula.

1.2. Análisis de `definitions.py`

Se incluye el asset `islas_raw` importándolo a través del módulo `test_checks.py` mediante la función `load_assets_from_modules`.

En cuanto a los checks que están en manifiesto, se incluye el check `check_estandarizacion_islas` del mismo módulo, cargándolo con la función `load_asset_checks_from_modules`.

1.3. Lanzamiento y materialización de los assets

Se lanzó el servidor local mediante el comando:

```
dagster dev -f definitions.py
```

Ejecución del check sin fallos

Para demostrar el funcionamiento correcto del sistema de checks, se ejecutó también una versión limpia del asset `islas_raw` sin la fila sucia. Al materializarla en la interfaz gráfica de Dagster, el check pasó exitosamente (aparece en verde), confirmando que todos los nombres de isla están estandarizados correctamente. Esto valida que el pipeline está preparado para evitar inconsistencias en la visualización y que el sistema de validación funciona como se esperaba.

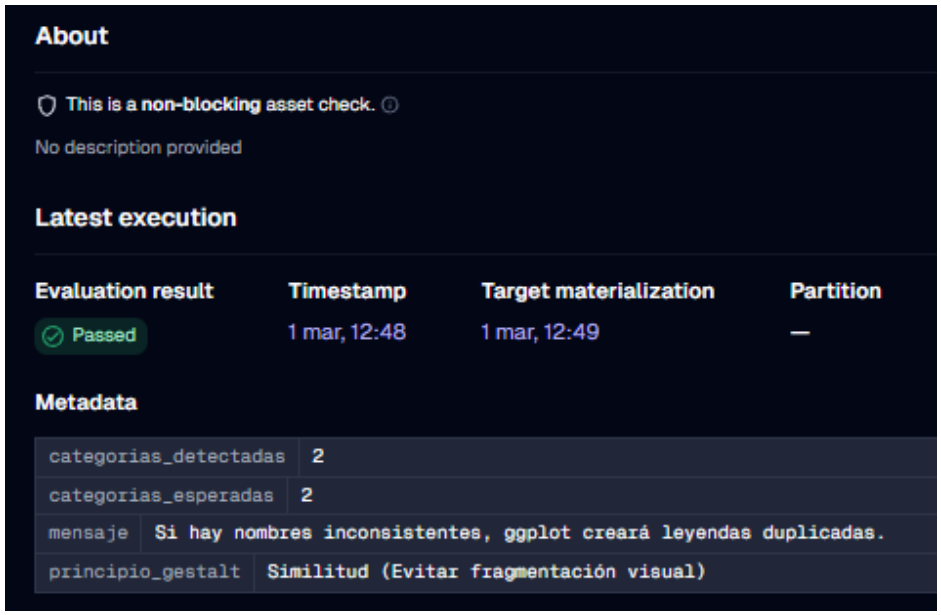

Figura 1.1: Interfaz de Dagster mostrando el check pasado correctamente.

Ejecución del check con fallo

Ahora, para demostrar que el check detecta correctamente los errores, se ejecutó la versión sucia del asset `islas_raw` con la fila sucia introducida, demostrando cómo el sistema detecta inconsistencias que generarían leyendas duplicadas en la posterior visualización. En la Figura 1.2 se muestra la interfaz de Dagster con el check fallido.

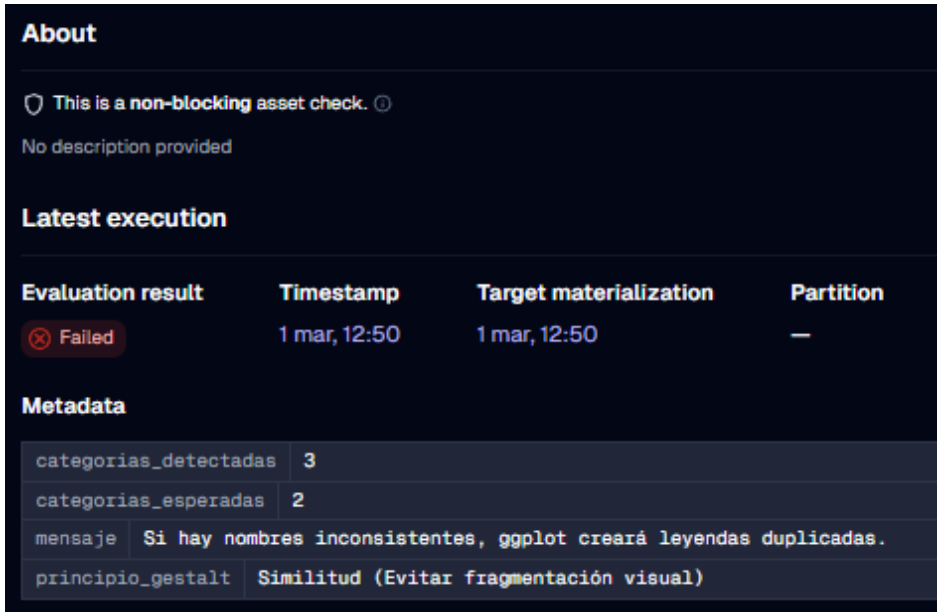

Figura 1.2: Interfaz de Dagster mostrando el check fallido.

2. Análisis de la práctica anterior

El proyecto implementa un pipeline DataOps con Dagster para analizar y visualizar la distribución de la renta en Canarias a partir del dataset oficial del ISTAC, integrando además información territorial y nivel educativo . A través de una arquitectura basada en assets interconectados, se estructura el flujo en tres fases: carga de datos (normalización de columnas, control de nulos y tipado), transformación (filtrado por municipios, agregaciones, cálculo de porcentajes y uniones) y visualización mediante la gramática de gráficos con plotnine (gráficos de barras, barras apiladas y scatter). El diseño sigue principios de calidad visual y perceptiva (orden por magnitud, control de cardinalidad, proporciones coherentes), incorporando checks que garantizan integridad, coherencia semántica y veracidad gráfica, lo que convierte el proyecto en un ejemplo sólido de integración entre ingeniería de datos, análisis estadístico y visualización analítica reproducible.

En la Tabla 2.1 se resumen los tres checks de calidad implementados en el pipeline, detallando su lógica de programación, su relación con los principios de diseño visual y la información que aportan a los metadatos para facilitar la interpretación de los resultados.

Cuadro 2.1: Matriz de Calidad – Checks implementados en el pipeline de rentas

Etapa	Nombre del Check	Descripción Técnica	Cómo programarlo (Lógica)	Relación con el Diseño / Gestalt	Información para Metadata
Carga (Raw)	`check_estandarizacion_islas`	Evita duplicados en la leyenda por inconsistencias de mayúsculas/minúsculas (ej. `"tenerife"` vs `"Tenerife"`).	`df["ISLA"].nunique()==df["ISLA"].str.capitalize().nunique()`	**Similitud:** evita que una misma isla se pinte con dos colores distintos en la leyenda del gráfico.	`categorias_detectadas, categorias_esperadas, mensaje`
Transformación (Curated)	`check_obs_value_rango`	Detecta valores de porcentaje fuera del rango válido ($0-100$) que indicarían datos corruptos en la carga.	`(df["OBS_VALUE"]>=0)&(df["OBS_VALUE"]<=100).all()`	**Proporcionalidad:** valores fuera de escala comprimen el eje y hacen que barras correctas parezcan insignificantes.	`valores_fuera_rango, min_valor, max_valor, mensaje`
Visualización (Asset)	`check_series_ordenadas`	Verifica que los municipios están ordenados ascendentemente por `OBS_VALUE` en el gráfico de barras.	`valores==sorted(valores)`	**Continuidad:** el ojo sigue una escalera suave en lugar de saltos erráticos, reduciendo el esfuerzo cognitivo.	`ordenado (bool), mensaje`

Resultados de la ejecución de los checks

Tras materializar el pipeline completo en Dagster, los tres checks de calidad definidos fueron ejecutados automáticamente. A continuación se muestran los resultados obtenidos en la interfaz web de Dagster.

Check 1 – Carga: `check_estandarizacion_islas`

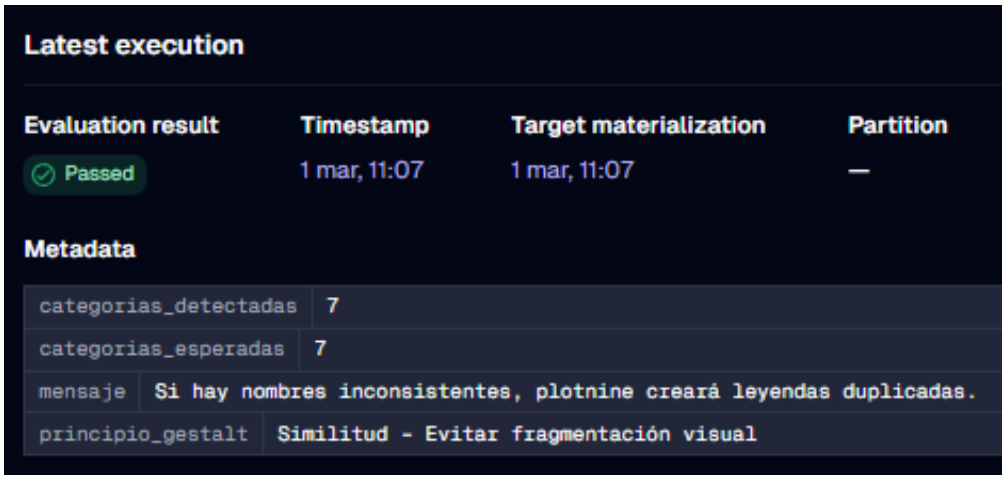

Figura 2.1: Resultado del check de estandarización de islas sobre `raw_codislas`.

El check ha **pasado** correctamente. Los metadatos confirman que el número de categorías detectadas y esperadas coincide (`categorias_detectadas = 7`, `categorias_esperadas = 7`), lo que indica que todos los nombres de isla en `codislas.csv` están escritos de forma consistente. No existe riesgo de que `plotnine` genere leyendas duplicadas en los gráficos.

Check 2 – Transformación: `check_obs_value_rango`

Figura 2.2: Resultado del check de rango de `OBS_VALUE` sobre `renta_municipios`.

El check ha **pasado** correctamente. Los metadatos muestran que el valor mínimo es `1.3` y el máximo `76`, ambos dentro del rango válido ($0-100$), y que el número de valores fuera de rango es `0`. Los datos de porcentaje sobre la renta total son coherentes y no presentan valores corruptos que pudieran distorsionar la escala del gráfico.

Check 3 – Visualización: `check_series_ordenadas`

Figura 2.3: Resultado del check de orden ascendente sobre `grafico_renta_municipios`.

El check ha **pasado** correctamente. El metadato `ordenado = Sí` confirma que los municipios están ordenados de menor a mayor por su valor de sueldos y salarios en el gráfico de barras. Esto garantiza que el principio de **Continuidad** se cumple: el ojo del lector puede seguir una escalera visual suave en lugar de encontrar saltos erráticos que aumenten el esfuerzo cognitivo necesario para interpretar el gráfico.

3. Verificación de los checks: casos de fallo

Para demostrar que los tres checks de calidad detectan correctamente los errores, se ha creado el fichero `scripts/casos_fallo.py`. Este fichero define tres assets alternativos — `raw_codislas_sucio`, `renta_municipios_sucia` y `renta_con_nombres_desordenada`— cada uno con su propio `@asset_check` asociado. En cada asset se introduce una **fila sucia** mediante `pd.concat` con datos incorrectos deliberados. El fichero incluye su propio bloque `Definitions` y se lanza de forma independiente al pipeline principal, sin modificar ningún asset de la práctica 2. A continuación se muestran los resultados obtenidos en la interfaz de Dagster tras materializar cada asset sucio.

Caso 1 – Fallo en `check_estandarizacion_islas`

El check ha **fallado** tal como se esperaba. Los metadatos muestran que el número de `categorias_detectadas` es **8**, mientras que el de `categorias_esperadas` tras normalizar con `str.lower().str.strip()` es **7**. En el campo `islas_unicas` puede observarse la causa directa: aparecen tanto `'Tenerife'` como `'tenerife'` como categorías distintas, lo que confirma que la fila sucia ha sido detectada. De no corregirse, `plotnine` generaría una entrada duplicada en la leyenda del gráfico, violando el principio de **Similitud** de la Gestalt.

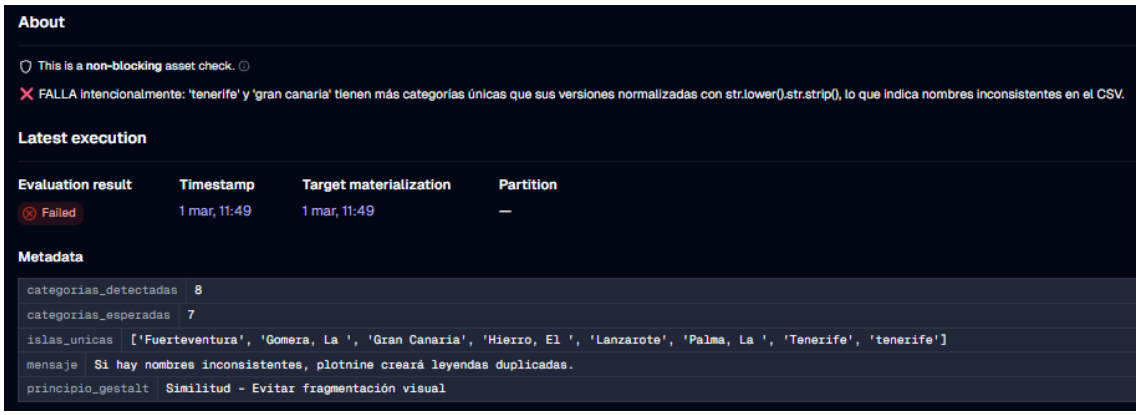

Figura 3.1: Resultado fallido del check de estandarización de islas sobre `raw_codislas_sucio`.

Caso 2 – Fallo en `check_obs_value_rango`

El check ha **fallado** tal como se esperaba. Los metadatos confirman que `valores_fuera_rango` es **1** y que el `min_valor` registrado es **-5**, correspondiente a la fila sucia introducida. El valor máximo sigue siendo **76**, dentro del rango válido. De no detectarse, el valor negativo comprimiría el eje Y del gráfico haciendo que las barras de los municipios correctos parecieran insignificantes, violando el principio de **Proporcionalidad**.

Figura 3.2: Resultado fallido del check de rango de `OBS_VALUE` sobre `renta_municipios_sucia`.

Caso 3 – Fallo en `check_series_ordenadas`

El check ha **fallado** tal como se esperaba. El metadato `ordenado` muestra el valor **No** y en `primeros_valores` puede verse la secuencia `[68.3, 57.2, 68.2, 64.8, 58.3]`, que no es ascendente. Esto confirma que la fila sucia con `OBS_VALUE = 999` concatenada al final ha roto el orden del DataFrame. De no corregirse, el gráfico presentaría saltos erráticos que aumentarían el esfuerzo cognitivo del lector, violando el principio de **Continuidad / Prägnanz** de la Gestalt.

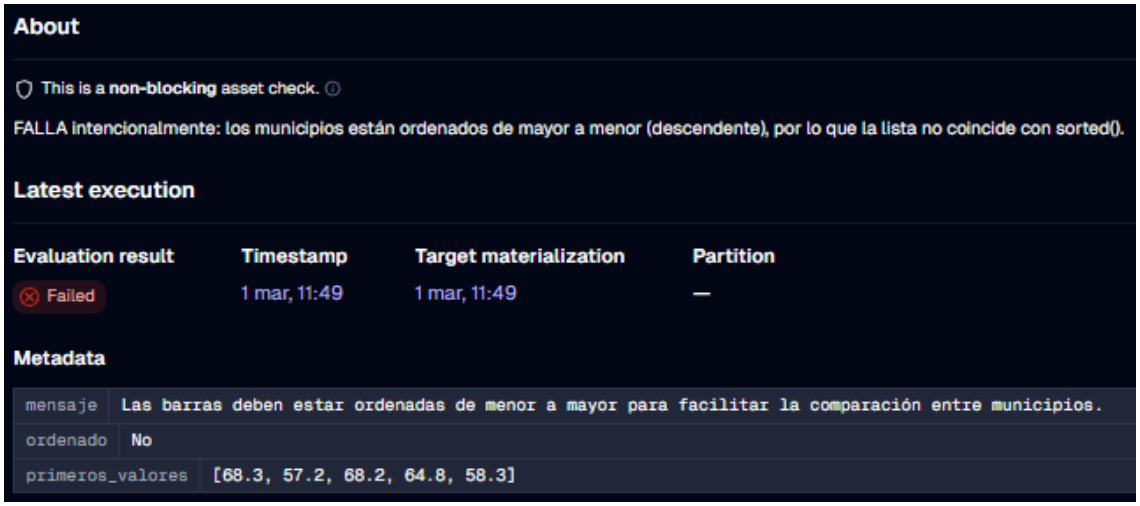

Figura 3.3: Resultado fallido del check de orden ascendente sobre `grafico_renta_municipios_desordenado`.

4. Enlace al repositorio de GitHub

https://github.com/feichay10/Practica_Gramatica_Graficos_DataOps